

자체 EAI 플랫폼

아키텍처 설계 문서

Enterprise Application Integration Platform — Architecture Design

FastMCP · React Flow · Debezium · AWS EC2/Docker

Version 0.1 (Draft)

작성일: 2026-07-15 · 작성자: doyoung.kang

문서 정보

항목	내용
문서명	자체 EAI 플랫폼 아키텍처 설계 문서
버전	0.1 (Draft) — 초기 설계 검토용
대상 독자	아키텍트, 백엔드/프론트엔드 개발자, 인프라(DevOps) 담당자
범위	논리·물리 아키텍처, 커넥터·CDC·파이프라인 엔진·UI 설계, AWS 배포
배포 환경	AWS EC2 + Docker (컨테이너 오케스트레이션)
핵심 스택	FastMCP(백엔드), React+React Flow(UI), Debezium(CDC), PostgreSQL/Redis/Kafka

목차

1. 개요

1.1 배경 및 목적

본 문서는 사내에서 자체 구축하는 EAI(Enterprise Application Integration) 플랫폼의 아키텍처를 정의한다. 이 플랫폼의 목적은 이기종 저장소(관계형 DB, NoSQL, SAP 등)에 흩어진 데이터를 표준화된 방식으로 수집하여, 목적 저장소(데이터베이스 또는 Amazon S3)에 안정적으로 적재하는 것이다. 사용자는 코드 작성 없이 웹 UI에서 드래그앤드롭으로 데이터 파이프라인을 구성하고, 이를 배치(스케줄) 방식으로 실행할 수 있어야 한다.

경쟁·참조 제품으로는 n8n, AWS Glue, Azure Data Factory가 있으며, 파이프라인 저작 경험(UX)은 특히 n8n의 노드 기반 캔버스에 가깝게 설계한다.

1.2 핵심 요구사항 요약

- **다양한 수집 방식:** ① 배치 DB→DB/S3 수집, ② SAP RFC 수집, ③ Debezium 기반 CDC 연결
- **지원 소스 DB:** MySQL, MSSQL, PostgreSQL, MongoDB (+ SAP ECC/S4HANA via RFC)
- **목적 저장소:** 관계형/NoSQL DB 및 Amazon S3(파일/오브젝트)
- **파이프라인 저작:** 웹 기반 드래그앤드롭 노드 에디터(n8n 스타일 UX)
- **실행 방식:** 스케줄 기반 배치 실행 + CDC 상시 스트리밍
- **백엔드:** FastMCP 기반 (도구/리소스 추상화, Streamable HTTP 전송)

1.3 용어 정의

용어	설명
EAI	이기종 애플리케이션·데이터 저장소를 통합 연계하는 아키텍처/플랫폼
커넥터(Connector)	특정 소스/타겟과의 연결·읽기·쓰기를 담당하는 플러그인 모듈
파이프라인(Pipeline)	소스→변환→타겟으로 이어지는 노드들의 방향성 그래프(DAG)
CDC	Change Data Capture. DB 변경(INSERT/UPDATE/DELETE)을 실시간 캡처하는 기법
RFC	Remote Function Call. SAP 시스템의 함수 모듈을 원격 호출하는 프로토콜
FastMCP	Python 기반 MCP(Model Context Protocol) 서버 구축 프레임워크
Worker	파이프라인의 실제 수집·변환·적재 작업을 수행하는 실행 프로세스

2. 시스템 목표 및 요구사항

2.1 기능 요구사항

구분	요구사항
FR-1 연결 관리	DB/SAP/S3 등 연결 정보(Connection) 등록·검증·시크릿 저장
FR-2 파이프라인 저작	드래그앤드롭 캔버스에서 노드 추가·연결·설정, 저장/버전 관리
FR-3 배치 실행	Cron 스케줄 및 수동 실행, 증분(incremental)·전체(full) 적재 모드
FR-4 SAP 수집	RFC/BAPI 및 RFC_READ_TABLE을 통한 테이블·함수 데이터 추출
FR-5 CDC 연결	Debezium 커넥터 설정·관리, 변경 이벤트의 타깃 적재
FR-6 데이터 변환	매핑·필터·타입 변환·조인 등 경량 변환 노드 제공
FR-7 모니터링	실행 이력, 성공/실패, 처리 건수, 로그, 재시도, 알림

2.2 비기능 요구사항

항목	목표
확장성	Worker 수평 확장으로 처리량 증대(큐 기반 분산 실행)
신뢰성	체크포인트/오프셋 기반 재시작, at-least-once 보장, 멍든 적재 권장
보안	연결 시크릿 암호화 저장, RBAC, 통신 TLS, 감사 로그
관측성	구조화 로그, 메트릭(CloudWatch), 실행 추적, 실시간 진행상황
운영성	컨테이너 기반 배포, 무중단 배포 지향, 설정의 코드화
성능	배치 대용량 처리 시 청크/병렬 처리, CDC 저지연 스트리밍

3. 전체 아키텍처

플랫폼은 관심사 분리를 위해 계층형(layered) 구조로 설계한다. 프레젠테이션(UI) → API/BFF(FastMCP) → 오케스트레이션·실행 → 커넥터 → 저장소의 5개 계층과, 파이프라인 정의·스키마·실행 이력·CDC 오프셋을 보관하는 메타데이터 스토어로 구성된다.

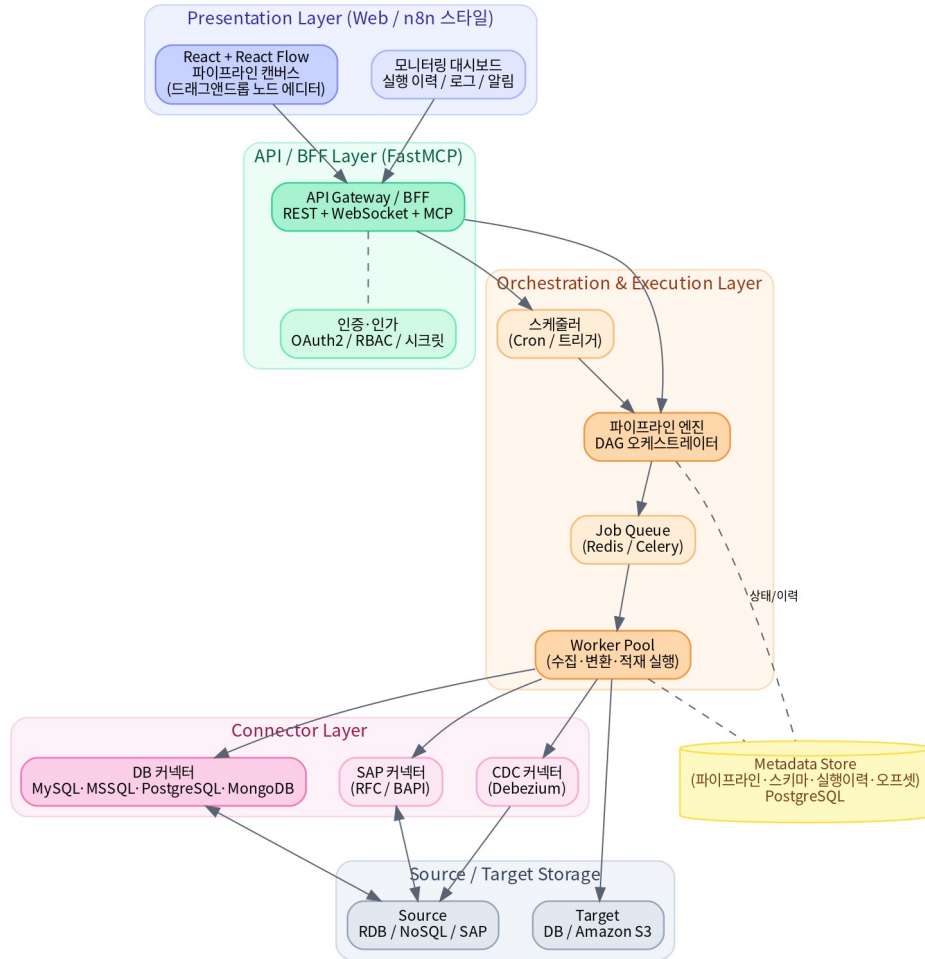


그림 1. 전체 논리 아키텍처 (계층 구조)

3.1 계층별 책임

계층	핵심 컴포넌트	책임
Presentation	React + React Flow, 대시보드	파이프라인 저작 캔버스, 모니터링/로그 시각화
API / BFF	FastMCP, 인증·인가	REST/WebSocket/MCP 엔드포인트, RBAC, 시크릿
Orchestration	엔진, 스케줄러, 큐	DAG 파싱, 스케줄 트리거, Job 분배
Execution	Worker Pool	수집(Extract)·변환(Transform)·적재(Load) 실행
Connector	DB/SAP/CDC 커넥터	소스·타겟 연결 추상화, 읽기/쓰기
Storage	Source/Target, Metadata	원천/목적 데이터, 파이프라인·실행 메타데이터

설계 원칙. 커넥터는 공통 인터페이스(read/write/test)를 구현하는 플러그인으로 만들어 신규 소스 추가 시 확장만으로 대응한다. 오케스트레이션과 실행을 분리(큐 경유)하여 부하에 따라 Worker만 늘리면 처리량이 선형으로 증가한다. 상태·이력은 모두 메타데이터 스토어에 남겨 재시작과 감사가 가능하도록 한다.

4. 데이터 수집 방식 상세

플랫폼은 세 가지 수집 패턴을 지원한다. 배치 DB 수집, SAP RFC 수집, 그리고 Debezium 기반 CDC이다.

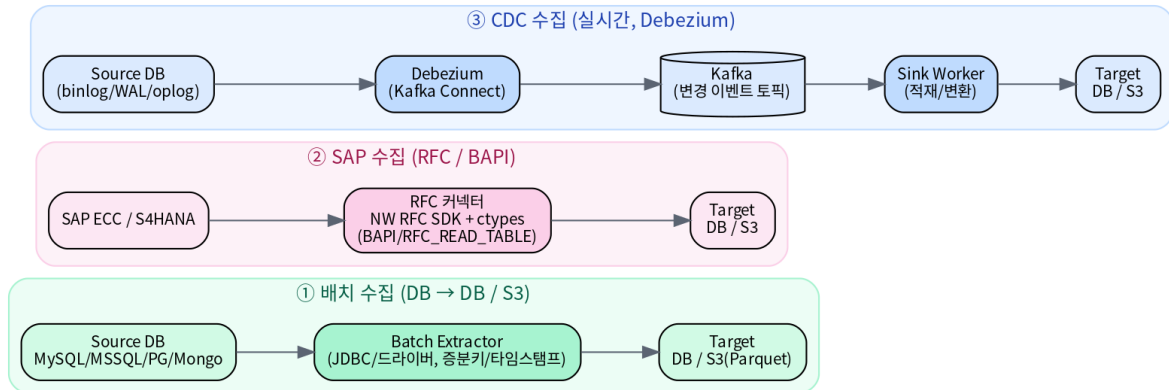


그림 2. 수집 방식별 데이터 흐름 (배치 / SAP RFC / CDC)

4.1 배치 DB → DB / S3 수집

가장 기본이 되는 방식으로, 소스 DB에서 데이터를 조회하여 타깃 DB 또는 S3에 적재한다. 스케줄러가 지정된 주기로 파이프라인을 실행한다.

- **적재 모드:** 전체(full) 재적재 또는 증분(incremental) — 증분키(PK/updated_at)·워터마크 기준
- **대용량 처리:** 키 레인지/오프셋 기반 청크 분할, Worker 병렬 처리, 커서 스트리밍으로 메모리 절감
- **S3 적재:** Parquet/CSV/JSON 포맷, 날짜 파티셔닝(예: dt=YYYY-MM-DD), 멀티파트 업로드
- **역동성:** 타깃 DB는 upsert(MERGE) 권장, S3는 실행 단위 경로 분리로 재실행 안전성 확보

4.2 SAP RFC 수집

SAP는 표준 DB 접근 대신 RFC(Remote Function Call)로 함수 모듈을 호출하여 데이터를 추출한다. Python에서는 SAP NetWeaver RFC SDK를 사용한다.

- **연결:** ashost/sysnr/client/user/passwd(또는 SNC) 기반 접속, 커넥션 풀 관리
- **추출 방식:** 업무용 BAPI/커스텀 RFM 우선, 범용 조회는 RFC_READ_TABLE 사용

주의사항. RFC_READ_TABLE은 대량 추출용으로 공식 설계된 것이 아니며, 행 폭 512자 제한으로 넓은 테이블은 컬럼을 분할 조회해야 한다. 부하 시 커넥션이 끊길 수 있어 재연결·재시도 로직이 필수다. 또한 기존 PyRFC 라이브러리는 유지보수가 중단되어, 최신 구현은 NW RFC SDK를 직접 바인딩(ctypes 등)하는 방식이 권장된다.

- **커넥터 격리:** SDK(네이티브 라이브러리) 의존성 때문에 SAP 커넥터는 전용 컨테이너 이미지로 분리 운영

4.3 CDC (Debezium) 수집

소스 DB의 트랜잭션 로그(MySQL binlog, PostgreSQL WAL, MongoDB oplog, MSSQL CDC 테이블)를 읽어 변경 이벤트를 실시간 스트리밍한다. Debezium을 사용한다.

배포 형태 선택. Debezium은 ① Kafka Connect, ② Debezium Server, ③ Embedded Engine의 세 가지 실행 형태가 있다. 본 플랫폼은 자동 장애 복구·태스크 재분배·오프셋 durable 관리가 기본 제공되는 Kafka Connect 방식을 표준으로 채택한다.

실행 형태	특징	적합성
Kafka Connect	분산·자동 페일오버·오프셋 저장 내장	표준 채택 (권장)
Debezium Server	단일 컨테이너, Kafka 불필요, 외부 오케스트레이션 필요	경량/소규모 대안
Embedded Engine	앱에 라이브러리로 내장, 클러스터링 없음	특수 통합 케이스

- **흐름:** Source DB → Debezium(Kafka Connect) → Kafka 토픽 → Sink Worker → 타깃 DB/S3
- **보장:** 오프셋 기반 재시작, at-least-once 전달 → 타깃에서 멱등 처리로 중복 흡수
- **스키마 변경:** 스키마 히스토리 토픽으로 DDL 추적, 스키마 레지스트리 연동 권장

5. 커넥터 레이어 설계

모든 커넥터는 공통 인터페이스를 구현하는 플러그인이다. 신규 소스/타겟은 이 인터페이스만 구현하면 파이프라인에서 노드로 노출된다.

5.1 공통 커넥터 인터페이스

```
class BaseConnector(Protocol):
    def test_connection(self) -> HealthResult: ...
    def discover_schema(self) -> list[TableSchema]: ...
    def read(self, query: ReadSpec) -> Iterator[RecordBatch]: ... # 소스
    def write(self, batch: RecordBatch, mode: WriteMode) -> WriteResult: ... # 타겟
```

read는 커서/청크 기반 이터레이터로 스트리밍하여 대용량에서도 메모리를 일정하게 유지한다. write는 append/upsert/overwrite 모드를 지원한다.

5.2 지원 커넥터 및 드라이버

소스/타겟	유형	드라이버 / 라이브러리	비고
MySQL	RDB	SQLAlchemy + PyMySQL	배치 + CDC(binlog)
MSSQL	RDB	pyodbc / pymssql	배치 + CDC
PostgreSQL	RDB	psycopg (v3)	배치 + CDC(WAL, logical decoding)
MongoDB	NoSQL	PyMongo	배치 + CDC(oplog/change stream)
SAP	ERP	NW RFC SDK (ctypes 바인딩)	RFC/BAPI, 전용 컨테이너
Amazon S3	Object	boto3	타겟(Parquet/CSV/JSON)

커넥션 & 시크릿. 연결 정보는 메타데이터 DB에 저장하되, 비밀번호/키는 애플리케이션 레벨 암호화(KMS 연동) 후 저장한다. 커넥터별 커넥션 풀을 유지하여 재접속 비용을 절감한다.

6. 파이프라인 엔진 & 스케줄링

파이프라인은 노드(수집·변환·적재)와 엣지로 구성된 DAG이다. 엔진은 이 DAG를 파싱하여 실행 계획을 만들고, 오케스트레이션과 실행을 분리한다. n8n의 큐 모드와 동일한 패턴으로, 메인(오케스트레이터)이 실행을 생성만 하고 Redis 큐를 통해 Worker가 실제 실행을 가져간다.

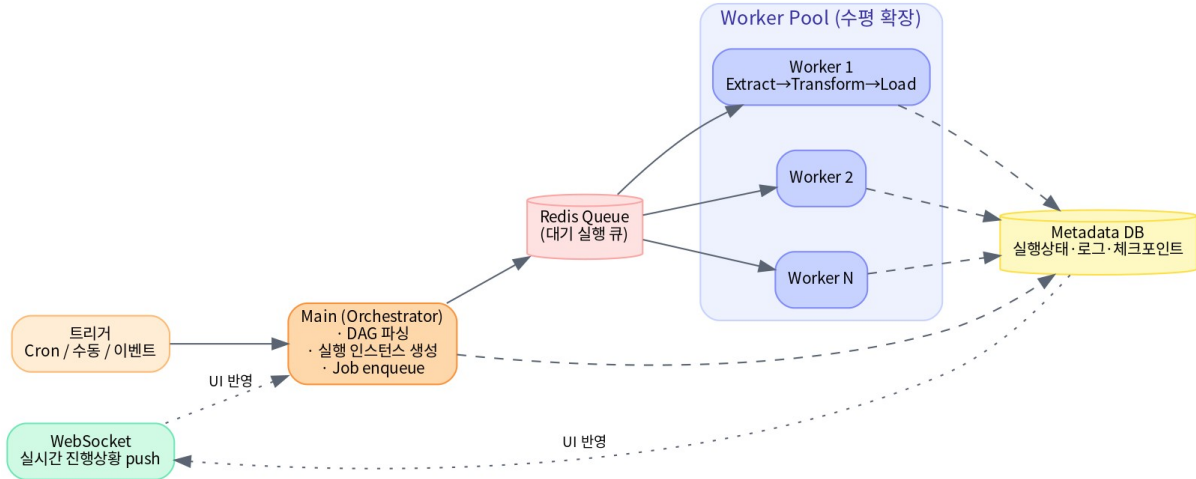


그림 3. 파이프라인 실행 아키텍처 (큐 기반 분산 실행)

6.1 실행 흐름

- 트리거(Cron/수동/이벤트)가 발생하면 오케스트레이터가 실행 인스턴스를 생성한다.
- 실행 ID를 Redis 큐에 enqueue하고, 상세 정의/상태는 메타데이터 DB에 기록한다.
- 가용한 Worker가 큐에서 Job을 가져와 DAG 노드를 순서대로 Extract→Transform→Load 실행한다.
- 진행 상황은 메타데이터 DB에 갱신되고, WebSocket으로 UI에 실시간 반영된다.
- 실패 시 노드 단위 재시도 정책(지수 백오프)과 체크포인트 기반 재시작을 적용한다.

6.2 스케줄러

- **Cron 표현식** 기반 주기 실행, 타임존 인식, 중복 실행 방지(락)
- **동시성 제어**: 파이프라인별 최대 동시 실행 수, 큐 우선순위 지정
- **백프레셔**: 큐 적체 시 Worker 오토스케일 트리거(메트릭 기반)

6.3 인프라 요건

큐 모드는 상태 공유를 위해 중앙 DB와 메시지 브로커가 필수다. 본 설계는 메타데이터/실행 상태에 PostgreSQL, 잡 큐에 Redis, CDC 이벤트 버스에 Kafka를 사용한다. (n8n 큐 모드가 SQLite를 지원하지 않고 PostgreSQL+Redis를 요구하는 것과 동일한 이유다.)

7. 프론트엔드 · n8n 스타일 UI/UX

저작 경험은 n8n의 노드 캔버스를 기준으로 설계한다. 좌측 노드 팔레트, 중앙 무한 캔버스, 우측 노드 설정 패널의 3분할 레이아웃을 채택한다.

7.1 핵심 구성

영역	구성 요소	설명
노드 팔레트	소스/변환/타겟 노드 목록	검색·드래그하여 캔버스에 배치
캔버스	무한 줌/팬, 노드·엣지	드래그앤드롭 연결, 스냅, 미니맵
설정 패널	노드별 파라미터 폼	연결 선택, 매핑, 쿼리, 스케줄
실행 오버레이	노드별 상태 뱃지	성공/실패/처리건수 실시간 표시
실행 이력	런(Run) 리스트, 로그	재실행, 상세 로그, 데이터 미리보기

7.2 기술 선택

- **캔버스 엔진:** React Flow(노드/엣지 렌더링·인터랙션의 사실상 표준). n8n도 유사한 커스텀 노드 캔버스를 사용
- **프레임워크:** React + TypeScript, 상태관리(Zustand/Redux), 폼(React Hook Form)
- **실시간:** WebSocket으로 실행 진행상황·로그 스트리밍
- **디자인:** n8n 유사 톤 — 밝은 캔버스, 컬러 코딩된 노드, 곡선 엣지, 카드형 설정 패널

8. 백엔드 (FastMCP) 설계

백엔드는 FastMCP를 기반으로 한다. FastMCP는 MCP 서버를 최소 보일러플레이트로 구축하는 Python 프레임워크로, 도구(tool)·리소스(resource) 추상화, 미들웨어, 인증 프로바이더, Streamable HTTP 전송을 제공한다. EAI의 각 기능(연결 테스트, 스키마 탐색, 파이프라인 실행 등)을 MCP 도구로 노출하면 UI뿐 아니라 LLM/에이전트에서도 재사용할 수 있다.

8.1 FastMCP 활용 포인트

- **도구(tool)로 기능 노출:** test_connection, discover_schema, run_pipeline, get_run_status 등
- **미들웨어:** 인증·로깅·레이트 리밋·감사 로그를 서버 오퍼레이션 인터셉트로 일괄 적용 (FastMCP 2.9+)
- **인증:** OAuth2 프로바이더(Google/GitHub/Azure/WorkOS 등) 및 커스텀 토큰 검증 지원
- **전송:** Streamable HTTP(JSON-RPC 2.0 over HTTP)로 컨테이너/로드밸런서 환경에 적합
- **캐싱/저장:** 프로덕션 워크로드용 영속 저장·응답 캐싱(FastMCP 2.13+) 활용

보안 참고. FastMCP는 활발히 발전 중이므로 CVE 패치가 반영된 안정 버전으로 고정(pin)하고, 정기 업데이트 정책을 둔다.

8.2 API 표면(예시)

구분	엔드포인트/도구	용도
연결	POST /connections, tool:test_connection	연결 등록·검증
메타	GET /connections/{id}/schema	스키마 탐색
파이프라인	POST /pipelines, PUT /pipelines/{id}	저장·버전
실행	POST /pipelines/{id}/run, tool:run_pipeline	수동/스케줄 실행
모니터링	GET /runs, WS /runs/{id}/stream	이력·실시간 로그

9. 기술 스택 요약

영역	기술	선정 이유
프론트엔드	React, TypeScript, React Flow	노드 캔버스 표준, n8n 유사 UX 구현 용이
백엔드	Python, FastMCP	도구/리소스 추상화, 미들웨어·인증 내장
오케스트레이션	커스텀 엔진 + Celery(선택)	DAG 실행, 큐 기반 분산
메시지 큐	Redis	잡 큐, 저지연 브로커
이벤트 버스	Apache Kafka	CDC 이벤트 스트리밍, durable
CDC	Debezium (Kafka Connect)	다중 DB CDC, 자동 페일오버
메타데이터	PostgreSQL	파이프라인·실행 이력·오프셋 저장
SAP 연동	NW RFC SDK	RFC/BAPI 표준 연동
타깃 스토리지	RDB / Amazon S3	구조화 적재 및 데이터레이크
배포	Docker, AWS EC2	컨테이너 기반 이식성·운영성

10. AWS EC2 / Docker 배포 아키텍처

모든 컴포넌트를 컨테이너화하여 AWS EC2에 배포한다. 초기에는 단일/소수 EC2 위에서 Docker Compose로 구성하고, 규모 확장 시 ECS(EC2 런치타입)로 이전하여 Worker 오토스케일을 적용한다.

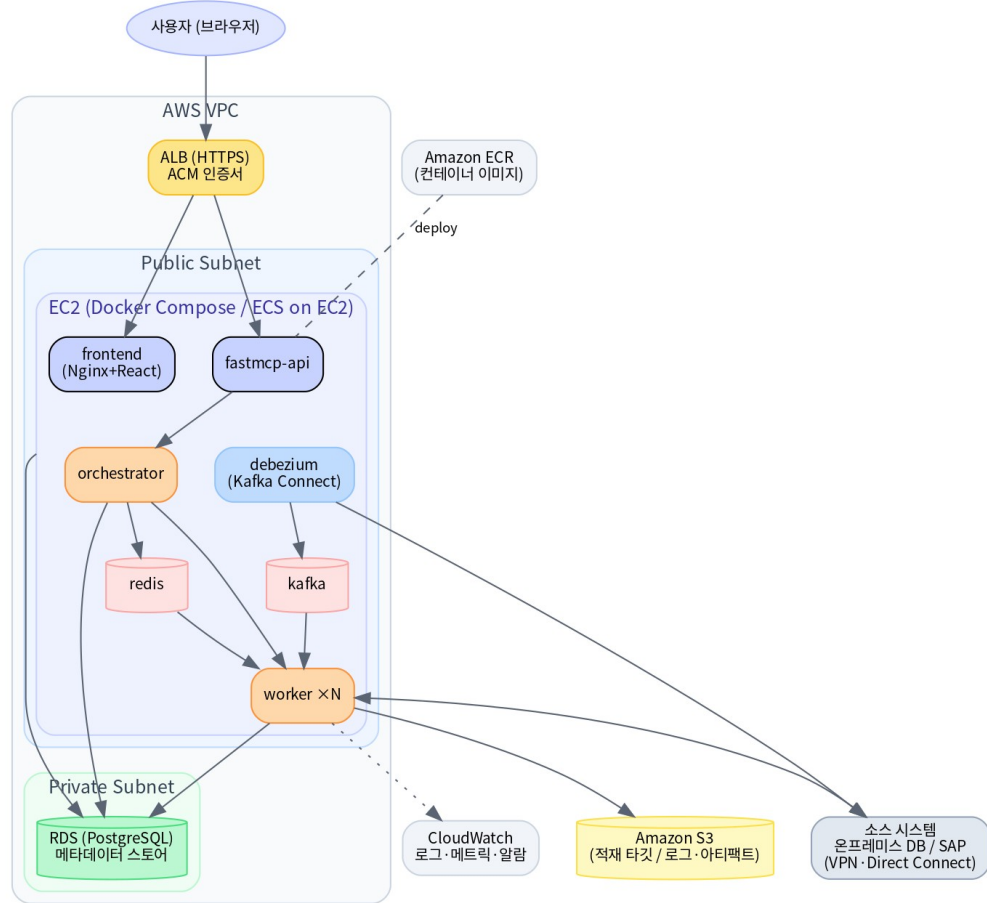


그림4. AWS EC2/Docker 배포 구성도

10.1 구성 요소

구성	역할	비고
ALB + ACM	HTTPS 종단, 라우팅	frontend/api 경로 분기
EC2 (Docker)	컨테이너 호스트	frontend·api·orchestrator·worker·debezium·redis·kafka
RDS (PostgreSQL)	메타데이터 스토어	Private Subnet, Multi-AZ 권장
Amazon S3	적재 타겟/로그·아티팩트	버킷 정책·수명주기
Amazon ECR	컨테이너 이미지 레지스트리	CI/CD 배포 파이프라인
CloudWatch	로그·메트릭·알람	Worker 오토스케일 트리거
VPN / Direct Connect	온프레미스 소스 연결	SAP·사내 DB 보안 연결

10.2 컨테이너 구성(개념)

services: frontend | fastmcp-api | orchestrator | worker (xN)

```
debezium(kafka-connect) | kafka | redis  
external: RDS(PostgreSQL) | S3 | ECR | CloudWatch
```

확장 전략. worker와 debezium은 상태를 외부(Redis/Kafka/RDS)에 두어 무상태로 유지하므로 수평 확장이 자유롭다. 초기 Docker Compose → 성장 시 ECS/EKS로 이전하는 경로를 열어둔다. Kafka/RDS는 운영 부담을 줄이기 위해 MSK/RDS 관리형 서비스로 대체하는 것을 검토한다.

11. 비기능 설계 (보안·확장성·관측성)

11.1 보안

- 연결 시크릿은 KMS 기반 암호화 후 저장, 애플리케이션 메모리 외 노출 금지
- RBAC: 역할(관리자/편집자/뷰어)별 파이프라인·연결 접근 제어
- 전 구간 TLS, Private Subnet 격리, 소스 연결은 VPN/Direct Connect
- 감사 로그: 연결 변경·실행·권한 변경 이력 기록

11.2 신뢰성 & 관측성

- 체크포인트/오프셋 기반 재시작, at-least-once + 멍등 적재로 정합성 확보
- 노드 단위 재시도(지수 백오프)와 DLQ(Dead Letter) 처리
- 구조화 로그 + CloudWatch 메트릭/알람, 실행 추적 대시보드
- 헬스체크·자동 재기동, RDS Multi-AZ, Kafka 복제 팩터 ≥ 2

12. 단계별 구현 로드맵

단계	범위	핵심 산출물
Phase 1 (MVP)	배치 DB→DB/S3 + 기본 UI	커넥터(MySQL/PG), 캔버스, 스케줄러, 실행 이력
Phase 2	커넥터 확장 + 변환	MSSQL/MongoDB, 변환 노드, RBAC, 모니터링 고도화
Phase 3	SAP RFC 수집	RFC 커넥터(전용 컨테이너), BAPI/RFC_READ_TABLE
Phase 4	CDC (Debezium)	Kafka Connect + Debezium, Sink Worker, 실시간 적재
Phase 5	운영 고도화	오토스케일, 관리형 서비스 전환, HA/DR, 감사

13. 리스크 및 고려사항

리스크	영향	완화 방안
SAP SDK 의존성/PyRFC 중단	SAP 커넥터 유지보수 리스크	NW RFC SDK 직접 바인딩, 전용 컨테이너 격리, 버전 고정
RFC_READ_TABLE 제약	대량/넓은 테이블 추출 한계	BAPI 우선, 컬럼 분할·재시도, CDS/추출기 검토
CDC 운영 복잡도	Kafka/Connect 운영 부담	MSK 등 관리형 채택, 모니터링·알람 강화
FastMCP 급속 변화	브레이킹 체인지·CVE	안정 버전 고정, 정기 업데이트·보안 패치 정책
대용량 배치 부하	소스 DB 부하·지연	증분 적재, 청크·병렬, 오프피크 스케줄

참고 자료

- FastMCP — Updates / Production features — <https://gofastmcp.com/v2/updates>
- Debezium — Architecture (Kafka Connect / Server / Engine) — <https://debezium.io/documentation/reference/stable/architecture.html>
- The Debezium Trio: Kafka Connect vs Server vs Engine — <https://blog.sequinstream.com/the-debezium-trio-comparing-kafka-connect-server-and-engine-runtimes/>
- n8n Docs — Configuring queue mode — <https://docs.n8n.io/hosting/scaling/queue-mode/>
- SAP Data Ingestion with Python (RFC) — <https://dlthub.com/blog/sap-data-ingestion-with-python-rfc>
- PyRFC is unmaintained — calling SAP RFC via nwrfsdk — <https://community.sap.com/t5/technology-q-a/pyrfc-is-unmaintained-calling-sap-rfc-directly-from-python-using-nwrfsdk/qaq-p/14305984>